Day 44 — API Specification

1. Scope

This document specifies the existing FastAPI application in `ats_engine/api.py`. It does not claim that the HR Interview AI Python modules have dedicated REST endpoints.

- API title: Zecpath ATS API
- Version: `1.0.0`
- Base path: `/api/v1`
- Request and response format: JSON unless an endpoint accepts file upload
- Framework: FastAPI with Pydantic request models

Interactive API documentation is provided by FastAPI when the application is running, normally at `/docs` and `/openapi.json`.

2. Endpoint Summary

Method	Path	Purpose
GET	`/api/v1/health`	Health check
POST	`/api/v1/resumes/upload`	Upload resume file and start background processing
POST	`/api/v1/resumes/parse`	Parse provided resume text
POST	`/api/v1/scoring`	Calculate an ATS score
POST	`/api/v1/shortlisting`	Shortlist candidates
GET	`/api/v1/jobs/{job_id}`	Retrieve asynchronous job status

3. Standard Response Contracts

Successful responses use the shape:

```
{
  "success": true,
  "message": "Operation completed",
  "data": {}
}
```

Error helper responses use the shape:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
```

```
    "message": "Description of the error"
  }
}
```

FastAPI/Pydantic validation failures may use FastAPI's standard 422 response body instead. Clients should not assume every framework-generated error has the custom error-helper shape.

4. Health Check

`GET /api/v1/health`

No request body is required.

A successful response returns HTTP 200 and includes `success: true` and `data.status: "healthy"`.

Example:

```
{
  "success": true,
  "message": "API is operational",
  "data": {
    "status": "healthy"
  }
}
```

The exact message and additional fields should be confirmed from the running implementation.

5. Resume Upload

`POST /api/v1/resumes/upload`

Content type: `multipart/form-data`.

Field:

- `file`: uploaded resume file.

The route creates an asynchronous processing job and returns job/resume identifiers or status information as implemented by the route. Use the returned job identifier with the job-status endpoint.

The in-memory stores are explicitly intended for demonstration and testing. They are not durable across process restarts and are not a production persistence solution.

6. Resume Parsing

`POST /api/v1/resumes/parse`

Content type: `application/json`.

Request model:

```
{
  "resume_text": "Data Scientist with Python, SQL and machine learning
experience."
}
```

Contract:

- `resume_text` is required.
- It must be a string with at least one character.
- An invalid request may produce HTTP 422.

The endpoint response includes a standard success envelope and a parsing status. Confirm detailed parsed fields against the current implementation before building a client that depends on them.

7. ATS Scoring

`POST /api/v1/scoring`

Content type: `application/json`.

Request model:

```
{
  "job_role": "Data Scientist",
  "scores": {
    "skill_match": 90,
    "experience_relevance": 80,
    "education_alignment": 75,
    "semantic_similarity": 85
  }
}
```

Validation:

- `job_role` is required and must be non-empty.
- All four score values are required.
- Each score must be between 0 and 100 inclusive.

The response includes the job role and calculated scoring results, including `overall_score` in the API tests. Use the implementation/configuration as the authority for the calculation and its weights.

8. Candidate Shortlisting

`POST /api/v1/shortlisting`

Content type: `application/json`.

Request model:

```
{
  "candidates": [
    {
      "candidate_id": "C001",
      "name": "Candidate A",
      "score": 91
    },
    {
      "candidate_id": "C002",
      "name": "Candidate B",
      "score": 78
    }
  ]
}
```

Each candidate requires:

- `candidate_id`: non-empty string.
- `name`: non-empty string.
- `score`: numeric value from 0 to 100 inclusive.

The response includes candidate counts and shortlisting results. The exact thresholds and categories are controlled by the shortlisting implementation; verify them before hard-coding client assumptions.

9. Asynchronous Job Status

`GET /api/v1/jobs/{job_id}`

Path parameter:

- `job_id`: identifier returned by resume upload.

The API contract lists these job states:

- `QUEUED`
- `PROCESSING`
- `COMPLETED`
- `FAILED`

An unknown job identifier is covered by the existing API tests and should be handled as a not-found condition.

The job store is in memory, so job status is not durable across application restarts.

10. Error Handling

Condition	Expected HTTP behavior
Successful request	2xx, normally 200
Invalid request model	422

Condition	Expected HTTP behavior
Unknown job	404
Unexpected internal failure	500

The `data/api_contracts.json` file defines `INVALID_REQUEST` as 400, `RESOURCE_NOT_FOUND` as 404, `VALIDATION_ERROR` as 422, and `INTERNAL_ERROR` as 500. Verify the exact code mapping in each route before relying on it in production clients.

11. HR Interview AI Integration Boundary

The current HR interview functionality resides in Python modules, including:

- `interview_ai/interview.py`
- `interview_ai/follow_up_engine.py`
- `scoring/hr_interview_scoring_engine.py`
- `scoring/interview_summary_generator.py`

This specification does not invent HTTP endpoints for these modules. A future REST integration should define explicit session IDs, request/response schemas, state handling, authentication, error behavior, and persistence before implementation.

12. Testing and Interactive Documentation

Relevant tests: `tests/test_ats_api.py`.

Start the API using the project's installed environment and ASGI server, for example:

```
python -m uvicorn ats_engine.api:app --reload
```

Then inspect `/docs` and `/openapi.json` locally. Do not expose a development server publicly without appropriate authentication and security controls.